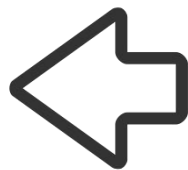


模块三：计算机视觉分类与检测

卷积算子的代数同构、ERF 统计力学与 Roofline 硬件极限

主讲人：利群他爹

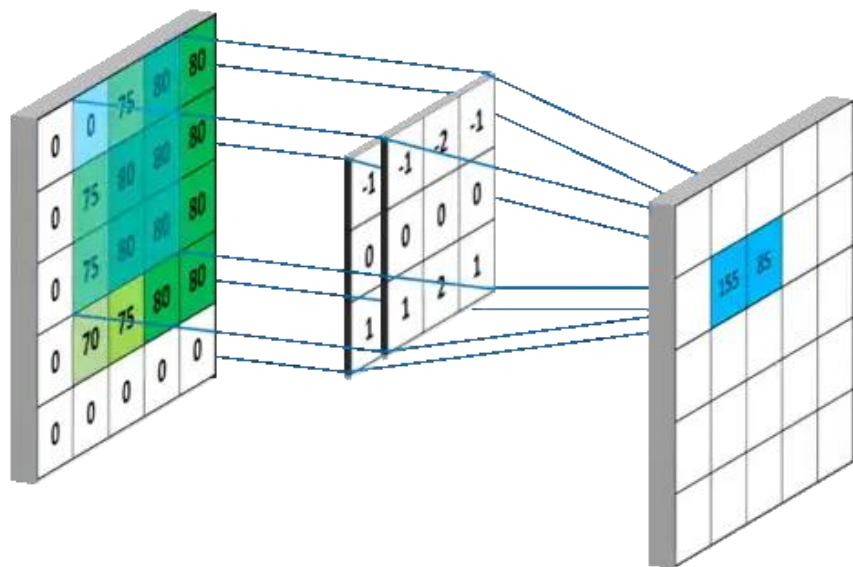


这是利群



卷积

- 何为卷积，第一印象maybe like：滤波器+点乘



教科书上一般定义函数 f, g 的卷积 $f * g(n)$ 如下:

连续形式:

$$(f * g)(n) = \int_{-\infty}^{\infty} f(\tau)g(n - \tau)d\tau$$

离散形式:

$$(f * g)(n) = \sum_{\tau=-\infty}^{\infty} f(\tau)g(n - \tau)$$

并且也解释了，先对 g 函数进行翻转，相当于在数轴上把 g 函数从右边褶到左边去，也就是卷积的“卷”的由来。

然后再把 g 函数平移*到 n ，在这个位置对两个函数的对应点相乘，然后相加，这个过程是卷积的“积”的过程。

$$(f * g)(n) = \int_{-\infty}^{\infty} f(\tau)g(n - \tau)d\tau$$

$$(f * g)(n) = \sum_{\tau=-\infty}^{\infty} f(\tau)g(n - \tau)$$

为什么“滑动窗口”是幻觉？

- GPU 有大量并行计算单元，但它的内存系统非常依赖规则、连续的访问模式。
- GPU 最擅长的是大规模规则矩阵乘法（GEMM）。矩阵乘法的数据访问模式高度规则，可以充分利用 cache、共享内存和 Tensor Core。因此，深度学习框架会把卷积改写成矩阵乘法，让硬件高效执行。

卷积运算满足线性性质：

$$(\alpha \mathbf{X}_1 + \beta \mathbf{X}_2) * \mathbf{K} = \alpha(\mathbf{X}_1 * \mathbf{K}) + \beta(\mathbf{X}_2 * \mathbf{K})$$

所以卷积可以表示为一个矩阵乘法： $\mathbf{y} = \mathbf{T}\mathbf{x}$ ，其中 $\mathbf{x}$ 是输入展平后的向量， $\mathbf{y}$ 是输出展平后的向量， $\mathbf{T}$ 是一个大矩阵。

Toeplitz 矩阵

一维卷积的 Toeplitz 矩阵

举一个简单的一维例子。假设输入信号 $\mathbf{x} = [x_0, x_1, x_2, x_3]$ ，卷积核 $\mathbf{k} = [k_0, k_1]$ ，步长 1，无 padding。输出长度为 3。

卷积运算：

$$\begin{aligned}y_0 &= k_0x_0 + k_1x_1 \\y_1 &= k_0x_1 + k_1x_2 \\y_2 &= k_0x_2 + k_1x_3\end{aligned}$$

写成矩阵形式：

$$\begin{bmatrix} y_0 \\ y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} k_0 & k_1 & 0 & 0 \\ 0 & k_0 & k_1 & 0 \\ 0 & 0 & k_0 & k_1 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

观察这个矩阵：每一行都是卷积核的平移，对角线上的元素相同（ k_0 在一条对角线上， k_1 在另一条对角线上）。这种矩阵叫 **Toeplitz 矩阵**。

二维卷积的块 Toeplitz 矩阵

二维卷积可以类似地展开。只不过矩阵的结构更复杂，被称为 **block-Toeplitz** 或 **Toeplitz-block-Toeplitz**。本质一样：一个稀疏、结构化的矩阵，其中每个非零块都对应卷积核的某个元素。

关键理解：卷积从来都是矩阵乘法

- 卷积是一个线性算子。
- 这个线性算子可以用一个矩阵表示。
- 这个矩阵不是任意的稠密矩阵，而是受“局部连接 + 权重共享”约束的**结构化稀疏矩阵**。

“局部连接”体现在矩阵中大部分位置为 0（因为每个输出只依赖局部输入）；“权重共享”体现在矩阵对角线元素重复（同一个卷积核权重在不同位置复用）。



im2col

- 虽然我们 know 卷积可以写成稀疏矩阵乘法，但直接使用稀疏矩阵乘法在 GPU 上效率不高，因为稀疏矩阵乘法的并行度和数据复用不如密集矩阵乘法。
- im2col 的思路是：把卷积的输入和卷积核重新排列，使卷积运算变成标准的密集矩阵乘法 (GEMM)，从而利用 GPU 对 GEMM 的高度优化。

im2col 的详细步骤

假设输入张量形状为 (B, C_{in}, H, W) ，卷积核形状为 $(C_{out}, C_{in}, k_h, k_w)$ ，输出形状为 $(B, C_{out}, H_{out}, W_{out})$ 。

1. 对于每个 batch 样本，我们将输入特征图划分为一个个卷积窗口（每个窗口大小为 $C_{in} \times k_h \times k_w$ ），将这些窗口展平成一个列向量，然后按顺序排列成一个大矩阵 $\tilde{\mathbf{X}}$ 。

具体形状：每个窗口展平后长度为 $C_{in} \cdot k_h \cdot k_w$ 。总共有 $B \cdot H_{out} \cdot W_{out}$ 个窗口（因为每个输出位置对应一个窗口）。因此：

$$\tilde{\mathbf{X}} \in \mathbb{R}^{(C_{in} k_h k_w) \times (B H_{out} W_{out})}$$

2. 卷积核展平：将卷积核张量重塑为形状 $(C_{out}, C_{in} k_h k_w)$ ，每一行对应一个输出通道的权重向量：

$$\tilde{\mathbf{K}} \in \mathbb{R}^{C_{out} \times (C_{in} k_h k_w)}$$

3. 矩阵乘法：

$$\tilde{\mathbf{Y}} = \tilde{\mathbf{K}} \cdot \tilde{\mathbf{X}}$$

得到的 $\tilde{\mathbf{Y}}$ 形状为 $(C_{out}, B H_{out} W_{out})$ 。

4. 重塑回输出：将 $\tilde{\mathbf{Y}}$ reshape 为 $(B, C_{out}, H_{out}, W_{out})$ 。

举个栗子

假设：输入 $1 \times 1 \times 3 \times 3$ ，卷积核 $1 \times 1 \times 2 \times 2$ ，stride=1, padding=0。输出大小 2×2 。

输入：

$$\mathbf{X} = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

卷积核：

$$\mathbf{K} = \begin{bmatrix} a & b \\ c & d \end{bmatrix}$$

输出：

$$\begin{aligned} y_{00} &= a \cdot 1 + b \cdot 2 + c \cdot 4 + d \cdot 5 \\ y_{01} &= a \cdot 2 + b \cdot 3 + c \cdot 5 + d \cdot 6 \\ y_{10} &= a \cdot 4 + b \cdot 5 + c \cdot 7 + d \cdot 8 \\ y_{11} &= a \cdot 5 + b \cdot 6 + c \cdot 8 + d \cdot 9 \end{aligned}$$

内存膨胀:im2col 显式复制了所有重叠的输入数据。
中间像素（如 5）出现在所有 4 个窗口中，被复制了 4 次。
角落像素（如 1）只出现 1 次。 $k \times k$ 卷积内存翻 k^2 倍

im2col 展开：四个窗口（按行优先顺序）：

- 窗口 (0,0): [1,2,4,5]
- 窗口 (0,1): [2,3,5,6]
- 窗口 (1,0): [4,5,7,8]
- 窗口 (1,1): [5,6,8,9]

展平后的 $\tilde{\mathbf{X}}$ 为 4×4 矩阵（每列一个窗口）：

$$\tilde{\mathbf{X}} = \begin{bmatrix} 1 & 2 & 4 & 5 \\ 2 & 3 & 5 & 6 \\ 4 & 5 & 7 & 8 \\ 5 & 6 & 8 & 9 \end{bmatrix}$$

卷积核展平为 1×4 ：

$$\tilde{\mathbf{K}} = [a, b, c, d]$$

矩阵乘法得到：

$$\tilde{\mathbf{Y}} = [a, b, c, d] \cdot \tilde{\mathbf{X}} = [y_{00}, y_{01}, y_{10}, y_{11}]$$

完美对应。

1.4 为什么 1×1 卷积特别高效?

1×1 卷积的本质

当 $k_h = k_w = 1$ 时, 卷积窗口大小就是 $C_{in} \times 1 \times 1$, 即每个窗口只包含单个像素的所有通道值。此时, `im2col` 不需要复制空间上的重叠数据, 因为窗口之间没有重叠。

输入特征图 $\mathbf{X}$ 形状 (B, C_{in}, H, W) , 可以直接 reshape 为 (B, HW, C_{in}) , 然后与卷积核 $\tilde{\mathbf{K}}$ 形状 (C_{out}, C_{in}) 做矩阵乘法。这个操作实际上就是对每个空间位置的通道向量做线性变换, 等价于全连接层 (但作用在每个位置上)。

硬件效率高的原因

- 没有内存膨胀: 不需要复制数据。
- 访存高度规则: reshape 后连续读取, 内存访问模式完美合并。
- 计算密度高: 对于每个输出位置, 计算 $C_{out} \times C_{in}$ 次乘加, 而访存只有 $C_{in} + C_{out}$ 个数据, FLOPs/Byte 很高, 倾向于 compute-bound (后面 Roofline 部分会详细分析)。



FFT视角下的卷积

傅里叶变换有一个极其重要的性质：空间域的卷积等于频域的逐点相乘。

一维形式：

$$\mathcal{F}\{f * g\} = \mathcal{F}\{f\} \cdot \mathcal{F}\{g\}$$

其中 $\mathcal{F}\{\cdot\}$ 表示傅里叶变换， $\cdot$ 表示逐点相乘。

二维形式完全类似：

$$\mathcal{F}\{\mathbf{X} * \mathbf{K}\} = \mathcal{F}\{\mathbf{X}\} \odot \mathcal{F}\{\mathbf{K}\}$$

其中 $\odot$ 是逐元素相乘（Hadamard product）。

2.1.2 这意味着什么？

卷积可以直接在空间域算（滑动窗口/GEMM），也可以绕道频域来算：

1. 对输入 $\mathbf{X}$ 做 FFT $\rightarrow \hat{\mathbf{X}}$
2. 对卷积核 $\mathbf{K}$ 做 FFT $\rightarrow \hat{\mathbf{K}}$
3. 在频域逐点相乘： $\hat{\mathbf{Y}} = \hat{\mathbf{X}} \odot \hat{\mathbf{K}}$
4. 对 $\hat{\mathbf{Y}}$ 做逆 FFT $\rightarrow \mathbf{Y}$

2.1.3 为什么频域路径可能更快？

关键在于复杂度分析。

假设输入大小 $H \times W$ ，卷积核大小 $k \times k$ 。

直接卷积（空间域滑动）的复杂度约为：

$$O(H \cdot W \cdot k^2)$$

FFT 路径的复杂度取决于 FFT 的复杂度 $O(N \log N)$ ，其中 N 是信号长度。

具体来说，需要做：

- 对输入做 FFT： $O(HW \log(HW))$
- 对卷积核做 FFT： $O(k^2 \log k^2)$ （通常可忽略，因为卷积核小）
- 频域逐点相乘： $O(HW)$
- 逆 FFT： $O(HW \log(HW))$

总复杂度约：

$$O(HW \log(HW))$$

FFT与纯卷积性能对比

2.1.4 FFT 路径的代价

对比：

- 直接卷积： $O(HWk^2)$
- FFT： $O(HW \log(HW))$

当 $k^2 > \log(HW)$ 时，FFT 路径更划算。

换句话说：

- 卷积核较大（如 5×5 、 7×7 、 11×11 ），且特征图也较大时，FFT 路径的复杂度优势明显。
- 卷积核很小（如 3×3 ）时，FFT 的常数因子和额外的虚数运算可能反而让它不划算。

2.1.4 FFT 路径的代价

- FFT 涉及复数运算，而深度学习网络通常是实数运算。需要额外处理实部和虚部，增加开销。
- FFT 有常数因子较大：即使复杂度低，对于小规模问题，实际运行时间可能更差。
- FFT 通常需要零填充卷积核到与输入相同大小，增加内存占用。

所以 FFT 路径主要适用于：大卷积核、大特征图、实数转复数带来的额外成本可以被复杂度优势覆盖的场景。



加速算法Winograd：小卷积核的乘法削减术

2.2.1 核心思想

对于小卷积核（尤其是 3×3 ），Winograd 算法通过巧妙的数学变换，减少乘法的次数，从而加速卷积。

为什么减少乘法很重要？

- 在硬件上，乘法比加法更昂贵（能耗更高、延迟更长）。
- 减少乘法次数，即使增加了加法次数，整体上往往能提速。

2.2.2 具体例子：F(2×2, 3×3)

这是最经典的 Winograd 卷积配置：

- 输入块大小： 4×4
- 卷积核大小： 3×3
- 输出块大小： 2×2

直接卷积的乘法次数：

- 每个输出像素需要 $3 \times 3 = 9$ 次乘法
- 总共 $2 \times 2 = 4$ 个输出像素
- 总计： $4 \times 9 = 36$ 次乘法

Winograd 的乘法次数：

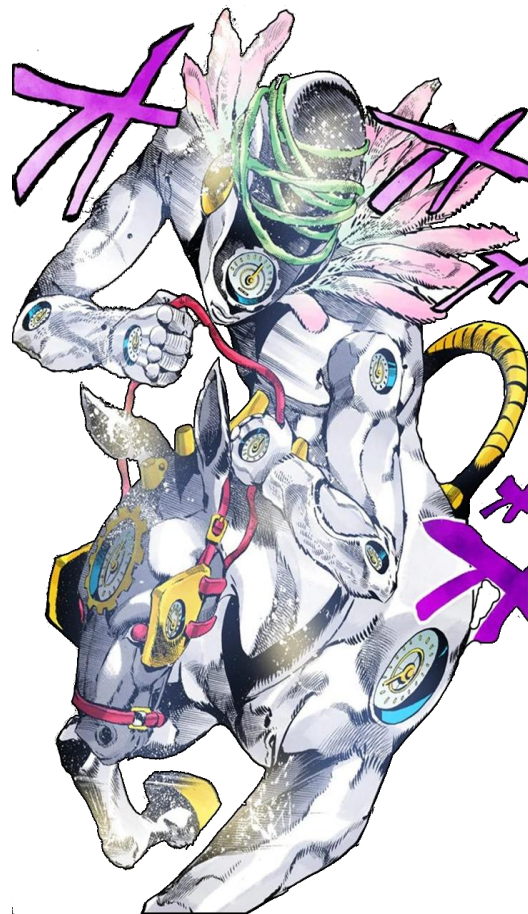
- 通过变换，将乘法次数降到 16 次
- 加速比： $36/16 = 2.25$ 倍

Winograd 算法将卷积分解为三个步骤：

$$\mathbf{Y} = \mathbf{A}^T [(\mathbf{GKG}^T) \odot (\mathbf{B}^T \mathbf{X} \mathbf{B})] \mathbf{A}$$

其中：

- $\mathbf{B}$ ：输入变换矩阵
- $\mathbf{G}$ ：卷积核变换矩阵
- $\mathbf{A}$ ：输出变换矩阵
- $\odot$ ：逐元素相乘



具体看winograd

- 先看一维

2.2.3 Winograd 的数学结构：从一维到二维

1. 一维 Winograd: F(m, r) 的最小滤波算法

先看一维卷积。假设我们要计算一个长度为 r 的卷积核 g 在一个长度为 $m + r - 1$ 的输入段 d 上滑动, 产生 m 个输出。直接计算需要 $m \times r$ 次乘法。

Winograd 的思想是：通过预先计算一些中间值，用更少的乘法得到同样的结果。

设输入向量 $d = [d_0, d_1, d_2, d_3]^T$, 卷积核 $g = [g_0, g_1, g_2]^T$, 输出 $y = [y_0, y_1]^T$ 。

Winograd 算法可以表示为三个变换：

$$y = A^T [(Gg) \odot (B^T d)]$$

其中：

- B^T 是输入变换矩阵，将输入 d 变换到 Winograd 域（4 个中间值）。
- G 是卷积核变换矩阵，将卷积核 g 变换到 Winograd 域（4 个中间值）。
- $\odot$ 是逐元素相乘（Hadamard product），对应 4 次乘法。
- A^T 是输出逆变换矩阵，将逐元素相乘的结果变换回输出 y 。

直接卷积：

$$\begin{aligned} y_0 &= g_0 d_0 + g_1 d_1 + g_2 d_2 \\ y_1 &= g_0 d_1 + g_1 d_2 + g_2 d_3 \end{aligned}$$

共 $2 \times 3 = 6$ 次乘法。

Winograd F(2,3) 的做法：

1. 计算 4 个中间值：

$$\begin{aligned} m_0 &= (d_0 - d_2) \cdot g_0 \\ m_1 &= (d_1 + d_2) \cdot \frac{g_0 + g_1 + g_2}{2} \\ m_2 &= (d_2 - d_1) \cdot \frac{g_0 - g_1 + g_2}{2} \\ m_3 &= (d_1 - d_3) \cdot g_2 \end{aligned}$$

2. 然后组合：

$$\begin{aligned} y_0 &= m_0 + m_1 + m_2 \\ y_1 &= m_1 - m_2 - m_3 \end{aligned}$$

可以验证这与直接卷积等价。这里乘法只有 4 次（ m_0 到 m_3 各一次），比直接 6 次少。但加法变多了

这 4 次乘法正是 Winograd 的核心：乘法次数从 $m \times r = 6$ 降到 $m + r - 1 = 4$ 。

cuda算子选优

对于 F(m×m, r×r):

- 直接卷积乘法次数: m^2r^2
- Winograd 乘法次数: $(m + r - 1)^2$

例如:

- F(2×2, 3×3): 直接 36, Winograd 16, 加速 2.25×
- F(4×4, 3×3): 直接 144, Winograd 36, 加速 4×
- F(6×6, 3×3): 直接 324, Winograd 64, 加速 5.06×

可以看到, 输出块越大, 加速比越高, 但数值稳定性和内存开销也越大。

2.3.1 四种主要路径

cuDNN 在实现卷积时, 主要有四种路径:

路径	核心思想	适用场景	优势	劣势
Direct	直接空间域卷积, 手工优化 CUDA kernel	小卷积核、小通道数	无额外内存开销	访存不规则, 效率有限
GEMM (im2col)	通过 im2col 转为矩阵乘法	中等规模、需要高吞吐	规则访存, Tensor Core 友好	内存膨胀大
FFT	频域逐点相乘	大卷积核、大特征图	复杂度低	复数运算、内存开销
Winograd	变换域乘法削减	3 × 3 小卷积核	减少乘法次数	数值精度风险、加法多

2.4 复杂度对比总结

假设输入 $H \times W$, 卷积核 $k \times k$, 输出通道 C_{out} , 输入通道 C_{in} 。

算法	近似复杂度	内存开销	最佳场景
Direct	$O(HWk^2C_{in}C_{out})$	低	小规模
GEMM/im2col	$O(H_{out}W_{out}k^2C_{in}C_{out})$	高 (k^2 倍膨胀)	中等规模, Tensor Core
FFT	$O(HW \log(HW)C_{in}C_{out})$	中 (复数)	大 k , 大 HW
Winograd	$O((m + r - 1)^2C_{in}C_{out})$	中	3 × 3 小卷积核

从图像到感受野

- 感受野 (Receptive Field) 是卷积神经网络中一个核心概念。它描述的是：输出特征图上的一个像素点，对应到原始输入图像上的区域大小。换句话说，就是“这个输出像素能看到输入图像的哪一块”。

想象一个卷积神经网络，输入是一张图片，经过若干卷积层后得到输出特征图。输出特征图上每一个像素，都是由前面某一层的一个局部区域计算得来的。这个局部区域再往前追溯，最终对应到输入图像上的一个区域，这个区域就是该输出像素的感受野。

例如：

- 一个 3×3 卷积层，输出像素的感受野是输入图像上的 3×3 区域。
- 如果堆叠两个 3×3 卷积层（步长1），那么第二层输出像素的感受野会扩大到输入图像上的 5×5 区域。

理论感受野

- 理论感受野是根据网络结构严格计算出的最大可能区域。它假设信息可以从输入区域的任何位置无衰减地传递到输出。

$$j_l = j_{l-1} \cdot s_l$$

$$r_l = r_{l-1} + (k_l - 1) \cdot j_{l-1}$$

其中：

- j_l ：第 l 层特征图上相邻像素在输入上的距离 (jump)
- r_l ：第 l 层每个像素的理论感受野
- k_l ：第 l 层卷积核大小
- s_l ：第 l 层步长

初始： $j_0 = 1, r_0 = 1$ (输入层每个像素感受野为1) 。

例如，两个 3×3 卷积 (步长1) 堆叠：

- 第1层： $r_1 = 1 + (3 - 1) \cdot 1 = 3$ (感受野 3×3)
- 第2层： $r_2 = 3 + (3 - 1) \cdot 1 = 5$ (感受野 5×5)

理论感受野是网络结构决定的“最大可见范围”。

那么问题来了，为什么会有jump

感受野递推

- Jump_l 第 l 层特征图上相邻两个像素，对应回输入图像上的距离（步长累积）

初始时，输入层（第 0 层）的像素就是原始图像像素：

$$j_0 = 1, \quad r_0 = 1$$

表示相邻像素距离为 1，每个像素的感受野就是它自己。

3.1.2 递推公式的推导

假设第 l 层由第 $l - 1$ 层经过一个卷积层得到，该卷积层的核大小为 k_l ，步长为 s_l 。

Jump 的递推

第 l 层特征图上相邻两个像素，是由第 $l - 1$ 层上相隔 s_l 个像素的两个位置卷积得到的（因为步长为 s_l ）。而在第 $l - 1$ 层上，相邻像素对应输入图像的距离是 j_{l-1} 。所以第 l 层相邻像素在输入图像上的距离就是：

$$j_l = j_{l-1} \cdot s_l$$

这很好理解：步长放大多少倍，jump 就放大多少倍。

Receptive field 的递推

第 l 层的一个像素，它由第 $l - 1$ 层上 $k_l \times k_l$ 的区域卷积得到。这 k_l 个像素在第 $l - 1$ 层上覆盖的范围是 k_l 个像素，但相邻像素之间在输入图像上的距离是 j_{l-1} 。所以这 k_l 个像素在输入图像上覆盖的“跨度”是 $(k_l - 1) \cdot j_{l-1}$ 。

而第 $l - 1$ 层每个像素自己的感受野是 r_{l-1} ，所以第 l 层一个像素的总感受野是：

$$r_l = r_{l-1} + (k_l - 1) \cdot j_{l-1}$$

这个公式的意思是：在上一层感受野的基础上，再扩展出由于本层卷积核大小和 jump 带来的额外区域。

假设输入图像 224×224 ，经过三个卷积层：

- 层 1: $k_1 = 3, s_1 = 1$
- 层 2: $k_2 = 3, s_2 = 2$
- 层 3: $k_3 = 3, s_3 = 1$

计算：

- 初始: $j_0 = 1, r_0 = 1$
- 层 1: $j_1 = 1 \cdot 1 = 1, r_1 = 1 + (3 - 1) \cdot 1 = 3$
- 层 2: $j_2 = 1 \cdot 2 = 2, r_2 = 3 + (3 - 1) \cdot 1 = 3 + 2 = 5$? 等一下，这里公式里用的是 j_{l-1} ，所以 $r_2 = r_1 + (k_2 - 1) \cdot j_1 = 3 + 2 \cdot 1 = 5$ 。但实际上因为步长 2，应该更大？我们仔细核对：层 2 的每个输出像素对应层 1 上 3×3 区域，但层 1 上相邻像素在输入上的距离是 1（因为层 1 步长 1），所以 3×3 区域在输入上覆盖 3 个像素的跨度，加上层 1 每个像素的感受野 3，总共是 $3 + 2 = 5$ 。这是正确的，虽然层 2 步长 2 但感受野大小不是由步长直接决定，而是由上一层 jump 决定。实际上 $j_2 = 2$ ，所以层 2 相邻像素在输入上距离 2。
- 层 3: $j_3 = 2 \cdot 1 = 2, r_3 = 5 + (3 - 1) \cdot 2 = 5 + 4 = 9$

有限感受野

3.2.1 理论感受野的“陷阱”

理论感受野告诉你：输出层一个像素“理论上”可以看到输入图像的 $r_L \times r_L$ 区域。但实际上，这并不意味着该区域内的所有像素对输出的贡献是均等的。恰恰相反，**贡献分布高度集中在中心，向边缘迅速衰减。**

这就引出了**有效感受野 (Effective Receptive Field, ERF)** 的概念：实际对输出有显著影响的输入区域，通常只是理论感受野中间的一小部分。

3.2.2 为什么贡献分布会中心化？——路径叠加的视角

考虑一个深层 CNN 中，输出层某个像素 y 与输入层某个像素 x 之间的“连接强度”。这个连接强度可以通过反向传播的梯度 $\frac{\partial y}{\partial x}$ 来衡量。

从输入到输出，信息沿着许多路径传播。每条路径经过一系列卷积核的权重相乘。对于靠近中心的输入像素，存在更多条路径能够到达输出像素；而远离中心的像素，能到达输出像素的路径数量急剧减少。

直觉：在每一层，卷积核的局部连接使得信息只能在局部邻域内传播。一个输入像素要影响远处的输出，必须经过多次“局部跳跃”。而每一步都有可能被权重削弱或改变方向。层数越多，能够成功到达输出的路径越少，尤其对于边缘像素。

我们可以把从输入到输出的一条路径看作一个随机游走。每一步的步长由卷积核大小和步长决定。经过多层后，这些路径的分布趋于**正态分布**（中心极限定理）。因此，输入像素对输出像素的影响强度，近似服从二维高斯分布：

$$\text{ERF}(x, y) \approx \frac{1}{2\pi\sigma^2} \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

其中 (x, y) 是输入像素相对于理论感受野中心的偏移， σ 是有效感受野的“半径”参数，通常远小于理论感受野半径。

关键点：这个高斯分布不是均匀的，而是中心高、边缘低。所以即使理论感受野很大，真正对输出有显著贡献的输入区域只是中心那一小坨。

3.2.4 一个简单的路径计数例子

假设我们有一个非常深的网络，每层都是 3×3 卷积，步长 1，无 padding。输出层一个像素，它对应的输入区域理论上是 $(2L + 1) \times (2L + 1)$ ，其中 L 是层数。但是中间像素到输出像素的路径数远远多于边缘像素。

例如，对于中心像素，每一层都有很多种路径组合可以到达输出；而对于边缘像素，必须每一步都沿着特定方向传播，路径数非常少。路径数大致服从二项分布，层数很多时，二项分布趋于高斯分布。这就是高维数学根源。

来看成本

- FLOPs 是 Floating Point Operations 的缩写，表示浮点运算次数。在深度学习中，我们通常统计的是乘加运算（MAC, Multiply-Accumulate）的数量，有时也直接称为 FLOPs。

对于标准卷积，假设：

- 输入特征图： $H \times W \times C_{in}$
- 输出特征图： $H_{out} \times W_{out} \times C_{out}$
- 卷积核： $k \times k$

每个输出像素由 $k \times k \times C_{in}$ 次乘加得到（即 $k^2 C_{in}$ 次乘法和 $k^2 C_{in}$ 次加法，共 $2k^2 C_{in}$ 次浮点运算）。总输出像素数为 $H_{out} \times W_{out} \times C_{out}$ 。

因此总 FLOPs 近似为：

$$\text{FLOPs} \approx 2 \times H_{out} \times W_{out} \times k^2 \times C_{in} \times C_{out}$$

有时简写为：

$$\text{FLOPs} \approx H_{out} W_{out} k^2 C_{in} C_{out} \quad (\text{如果只算乘法})$$

或

$$\text{FLOPs} \approx 2 H_{out} W_{out} k^2 C_{in} C_{out} \quad (\text{乘加算两次})$$

但无论哪种，它只统计了“计算了多少次”，完全没有考虑数据从内存搬到计算单元的代价。



Roofline Model：硬件性能的天花板

4.2.1 两个硬件极限

任何处理器（CPU/GPU）都有两个基本极限：

1. **峰值算力** π ：单位时间内能执行的最大浮点运算次数（FLOPS/s），由计算核心数量和频率决定。
2. **峰值内存带宽** β ：单位时间内能从内存读取的最大数据量（Bytes/s），由内存类型和总线宽度决定。

这两个极限就像屋顶的两条边，实际性能不可能超过它们。

4.2.2 计算密度（Operational Intensity）

定义一个算子或一段代码的**计算密度** I ：

$$I = \frac{\text{FLOPs}}{\text{Bytes}}$$

分子是总共需要执行的浮点运算次数，分母是总共需要从内存（通常是 DRAM）读取和写回的数据字节数。

计算密度衡量了“每搬运一个字节的数据，能做多少次浮点运算”。如果 I 很大，说明这个算子计算密集，数据搬运相对较少；如果 I 很小，说明这个算子数据搬运频繁，计算很少。

实际能达到的性能（每秒浮点运算次数，FLOPS/s）受到两个约束：

- 不能超过峰值算力 π ： $\text{Perf} \leq \pi$
- 不能超过内存带宽允许的最大值：每秒最多搬运 β 字节，而每字节对应 I 次运算，所以每秒最多完成 $\beta \times I$ 次浮点运算： $\text{Perf} \leq \beta I$

因此：

$$\text{Perf} \leq \min(\pi, \beta I)$$

这个不等式就是 Roofline Model 的核心。

4.2.4 两个区域

- 当 I 很大（ $\beta I \geq \pi$ ）时，性能受限于算力，称为 **compute-bound**。此时优化计算效率（如使用 Tensor Core、减少指令开销）才有效。
- 当 I 很小（ $\beta I < \pi$ ）时，性能受限于带宽，称为 **memory-bound**。此时无论怎么优化计算，性能都不会提高，必须减少数据搬运或提高数据复用。

用一张图表示：横轴是计算密度 I （对数刻度），纵轴是性能（对数刻度），画出的曲线像一个屋顶——先随 I 线性上升（带宽限制），然后变平（算力限制）。这就是 Roofline Model 名称的由来。

